

Ihsan Salari

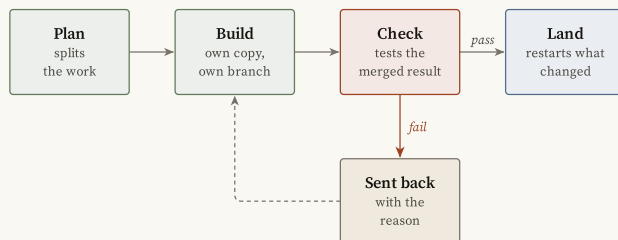
I build AI agents that do engineering work, and the checks that decide whether that work is accepted.

I'm a second-year electrical engineering student at Waterloo. Since March I've been building with Claude Code: agents that write and land software, design circuit boards and lay out chip blocks, and tools that turn my courses into lessons and notes. The pattern is the same in each. The agent does the work, deterministic tools check it, a failed check sends it back with the reason, and anything irreversible, like paying for a board, waits for a person.

autobox

<https://github.com/ihsan-sa/autobox>

A server that runs Claude Code on my projects without anyone watching. A planning session per project splits work into tracks. Each worker runs in its own git worktree as a loop of fresh-context iterations, keeping its state in a brief and a journal on disk. A hook commits its work, a rule table checks every command before it runs, and one program merges, only after the gates pass on the branch merged with current main. Technical note: [autobox.pdf](#)



hwde

<https://github.com/ihsan-sa/hwde>

An agent that turns a plain-English brief for a circuit board into the files a factory needs. Subagents make the design decisions, and Python scripts run every check that can be computed, from KiCad's design-rule check to ngspice simulation. A failed check becomes work orders for fixer agents. How well the router does feeds back into where parts are placed. The checkers are themselves tested against boards with planted faults. Two boards it designed have been made. Neither has been powered on yet. Technical note: [hwde.pdf](#)

chip-flow

<https://github.com/ihsan-sa/chip-flow>

The same approach for chip blocks, on open tools and the GF180MCU process. One engine sits under three skills: digital, analog and mixed-signal. Digital blocks get their testbench written before the RTL, and the testbench is itself tested with hidden tests and mutants. Analog layout is written as code that generates the layout, then checked with DRC, LVS and simulation of the extracted layout. Small analog blocks have gone from spec to a signed release record. Larger digital designs don't pass their gates yet, and nothing has been sent to fabrication. Technical note: [chip-flow.pdf](#)

lesson-builder

<https://github.com/ihsan-sa/lesson-builder>

A Claude Code skill that builds interactive web lessons for my courses, with a tutor inside each one. Specialist agents research, plan, draw and build demos. A person approves the plan once, and separate reviewers check the code, the teaching, the visuals and the science. The tutor can propose edits to the lesson while a student reads it, and an edit only lands after code review passes. I tested the teaching reviewer blind, with planted mistakes and the answer key withheld. Technical note: [lesson-builder.pdf](#)

pdf-material-builder

<https://github.com/ihsan-sa/pdf-material-builder>

A skill that turns a course into LaTeX study documents in one look and one voice. For a large build, agents extract each lecture in parallel into one conventions file that every writer reads first. Five reviewers then read the result at once, one of them re-deriving the math in SymPy, and a filter throws out the findings that don't survive a re-check. This document was built with it. Technical note: [pdf-material-builder.pdf](#)